Module 5 Week A — Integration Task: ML Evaluation Pipeline

This integration task brings together everything from Week A — data splitting, Pipelines, cross-validation, classification metrics, and regularization — into a single reproducible evaluation pipeline. You will systematically compare five model configurations (three real models + two dummy baselines) on the Petra Telecom churn dataset and produce a results table with a recommendation grounded in the business context.

This pipeline will be extended in Week B, where you will add decision trees, random forests, and advanced evaluation techniques to produce the final Module 5 model comparison deliverable.

Framing: Where the Week A Arc Lands

If you saw accuracy around 0.60 in the drill and lab, that's the setup for this task. Both used `class_weight="balanced"` on an imbalanced dataset (~16% churn), which trades majority-class accuracy for better recall on churners. This task builds the explicit baseline comparison that explains those numbers: you'll compare your logistic regression and ridge classifier against two `DummyClassifier` baselines and write a recommendation grounded in F1 rather than accuracy. Section 5 of the Week A reading is the conceptual foundation; this is the quantitative demonstration. The drill and lab left you with questions about why accuracy-focused metrics were misleading; this task is where those answers get built from actual numbers.

Learning Objectives

- Build a preprocessing pipeline with `ColumnTransformer` for mixed-type data
- Define and evaluate multiple model configurations programmatically
- Run stratified cross-validation across all configurations
- Produce a structured results table with multiple metrics
- Write a recommendation that connects model metrics to business context

---

Setup

1. **Accept the GitHub Classroom assignment:** Accept the Integration 5A assignment
2. **Clone your repo:**

```
git clone <your-repo-url>
cd <your-repo-name>
```

3. **Install dependencies:**

```
pip install -r requirements.txt
```

4. **Create a working branch:**

```
git checkout -b integration-5a-ml-pipeline
```

The starter file `evaluation_pipeline.py` contains function signatures with docstrings and TODO comments. The dataset is at `data/telecom_churn.csv`. Implement each function following the TODO directions, the task descriptions below, and the Week A reading.

Task 1: Load, Prepare, and Split Data

Load the telecom churn dataset. Identify which columns are numeric and which are categorical. Split into features (`X`) and target (`y` = `churned`). Then create an 80/20 train/test split with `train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)`.

Why split now if Task 4 uses cross-validation? The test set is held out for **Task 5** — a final evaluation of the selected best model on data it has never seen during model selection. Cross-validation in Task 4 runs on the training subset only. This mirrors the lab's train/test discipline and stacks the splitting skill from Lab 5A into a multi-model comparison context.

Task 2: Build the Preprocessing Pipeline

Create a `ColumnTransformer` that applies:

- `StandardScaler` to numeric features (tenure, monthly charges, total charges, `num_support_calls`, and any other numeric columns)
- `OneHotEncoder(drop="first", handle_unknown="ignore")` to categorical features (contract type, internet service, payment method, and any other categorical columns)

Wrap this in a `Pipeline` that pairs the preprocessor with a placeholder model (you will swap the model in the next task).

Task 3: Define Five Model Configurations

Create a dictionary or list of five model configurations, each paired with the preprocessing pipeline from Task 2:

Configuration	Model	Key Parameters
Logistic Regression (default)	`LogisticRegression`	`C=1.0`, `random_state=42`, `max_iter=1000`, `class_weight="balanced"`
Logistic Regression (L1, C=0.1)	`LogisticRegression`	`C=0.1`, `penalty="l1"`, `solver="saga"`, `random_state=42`, `max_iter=1000`, `class_weight="balanced"`
Ridge Classifier	`RidgeClassifier`	`alpha=1.0`, `random_state=42`, `class_weight="balanced"`
Most-frequent Dummy	`DummyClassifier`	`strategy="most_frequent"`
Stratified Dummy	`DummyClassifier`	`strategy="stratified"`, `random_state=42`

Two dummy baselines are included to teach two different lessons.

`DummyClassifier(strategy="most_frequent")` always predicts the most common class ("not churn" on this dataset). It demonstrates how accuracy can be inflated by a model that ignores the minority class entirely — look at its accuracy in your results table, it will beat the real models. Because the real models use `class_weight="balanced"`, they trade accuracy for recall on the minority class, so the dummy wins on accuracy. That comparison is the point of this baseline: accuracy alone is not a trustworthy metric on imbalanced data.

`DummyClassifier(strategy="stratified")` predicts randomly in proportion to the observed class frequencies. This is what random guessing actually looks like on this dataset — it tells you what your real models need to clearly beat on F1 if they've learned anything useful. If your best real model is only marginally better than the stratified dummy on F1, that tells you linear models have hit a ceiling on what they can learn from the current features. Module 5 Week B will return to this by introducing tree-based models, which often pick up patterns that linear models miss on tabular data.

Compare the real models against **both** dummies: real models should lose to the most-frequent dummy on accuracy (expected, because of `class_weight="balanced"`) and should clearly beat both dummies on F1. Your recommendation in Task 5 needs to address both comparisons.

Task 4: Run Cross-Validation and Collect Results

Use `X_train, y_train` only — not the full `X, y`. Cross-validation on the training subset keeps the test set completely unseen until Task 5.

For each of the five configurations:

1. Run 5-fold stratified cross-validation on `(X_train, y_train)` using `cross_validate` with `scoring=["accuracy", "precision", "recall", "f1"]`.
2. Record the mean and standard deviation for each metric.
3. Assemble the results into a table with this format:

Model	Mean Accuracy	Std	Mean Precision	Mean Recall	Mean F1
LogReg (default)	0.xxx	0.xxx	0.xxx	0.xxx	0.xxx
LogReg (L1, C=0.1)	0.xxx	0.xxx	0.xxx	0.xxx	0.xxx
RidgeClassifier	0.xxx	0.xxx	0.xxx	0.xxx	0.xxx
Most-frequent Dummy	0.xxx	0.xxx	0.xxx	0.xxx	0.xxx
Stratified Dummy	0.xxx	0.xxx	0.xxx	0.xxx	0.xxx

Print this table to the console. You may format it using f-strings, pandas DataFrame, or any readable format.

Task 5: Final Evaluation on the Held-Out Test Set

After comparing models with cross-validation, commit to a recommendation by evaluating the selected model on the test set that was held out in Task 1. This is the model's first and only encounter with this data — it is your honest, single-shot read of how the model would perform on truly unseen customers.

Implement `final_evaluation(pipeline, X_train, X_test, y_train, y_test)` in your starter. The function should:

1. Fit the pipeline on `(X_train, y_train)` (the full training data, not a CV fold).
2. Predict on `X_test`.
3. Compute accuracy, precision, recall, and F1 on the held-out predictions.
4. Return the four metrics as a dictionary with keys `"accuracy"`, `"precision"`, `"recall"`, `"f1"`.

Then in your `__main__` block, select the best real model from the Task 4 results (highest `f1_mean` among the non-dummy rows), look it up in the `models` dict, and call `final_evaluation` on it. Print the final test-set metrics alongside the CV estimates.

What to check: the test-set metrics should be close to the CV estimates. If the test-set F1 is substantially lower than the CV mean F1 (more than about one CV standard deviation), the CV estimates were optimistic and you should note this in your recommendation. If they match, that's evidence your CV procedure and the model's generalization are both working as expected.

Why this matters: Cross-validation gives you out-of-sample estimates, but every fold is still data the model family has "seen" during selection. The held-out test set is the only data that was never part of any CV fold during model selection — it's the single honest measurement.

Task 6: Write the Recommendation

Based on your results table and the final test-set evaluation, identify the best-performing configuration. Write a 1-paragraph recommendation (4–6 sentences) that explains:

- Which model you recommend and why
- Why accuracy alone is not sufficient for this problem (hint: churn detection has asymmetric costs – missing a churning customer is more expensive than a false alarm). Cite the most-frequent dummy's accuracy as evidence.
- How precision and recall trade off for the recommended model
- What the comparison against **both** dummy baselines reveals — the most-frequent dummy shows why accuracy is misleading on imbalanced data, and the stratified dummy shows what random guessing in proportion to class frequencies looks like. Your best real model should beat both dummies on F1 — but by how much? An F1 of ~0.35 vs a stratified baseline of ~0.17 is meaningful, but it also tells you the model catches churners at only roughly twice the rate of random guessing proportional to class frequencies. Address whether this is strong enough to deploy, or whether this is the ceiling of what a linear model can learn from these features — we'll return to this question in Module 5 Week B with tree-based methods.
- Whether the final test-set metrics from Task 5 confirm or complicate the CV-based recommendation

Include this recommendation as a docstring or block comment at the bottom of your script AND in your PR description.

Challenge Extensions

These are optional extensions for learners who want to go deeper. They are not graded and do not affect your integration score. No starter code is provided – design and implement these independently.

Tier 1: Per-Class Analysis

The default `cross_validate` with `scoring="precision"` computes precision for the positive class only. Extend your pipeline to compute per-class metrics:

- Run `cross_val_predict` to get out-of-fold predictions for the full training set.
- Generate a full `classification_report` from the aggregated out-of-fold predictions for each model.
- Identify which model performs best specifically on the minority class (churned customers) and whether this changes your recommendation.
- Discuss what happens to per-class recall when you switch from the default `LogisticRegression` to the L1-regularized version.

Tier 2: Pipeline Factory with Feature Engineering

Refactor your evaluation pipeline into a reusable module:

- Write a `build_pipeline(model, numeric_features, categorical_features)` factory function that constructs a `ColumnTransformer` + model pipeline from arguments.
- Add a feature engineering step inside the pipeline: create interaction features (e.g., `tenure * monthly_charges`) and polynomial features for the numeric columns using `PolynomialFeatures(degree=2, interaction_only=True)`.
- Re-run the model comparison with and without feature engineering.
- Document the results: does feature engineering improve any model? Does it hurt any model? Connect your findings to the bias-variance tradeoff.

Tier 3: Custom Cross-Validation Engine

Implement stratified k-fold cross-validation from scratch using only NumPy and pandas (no `cross_val_score`, no `StratifiedKFold`):

- Write a function that takes `X`, `y`, a model, and `k`, and returns an array of k scores.
- Your fold assignment must preserve class proportions (implement the stratification logic yourself).
- Handle edge cases: what if `k` does not divide evenly into the data? What if one class has fewer than `k` members?
- Validate your implementation by comparing its output to scikit-learn's `cross_val_score` on the same data with the same `random_state`.
- Write at least 3 pytest tests for your implementation covering: correct number of folds, non-overlapping test sets across folds, and preserved class ratios within each fold.

Submission

Open a pull request from `integration-5a-ml-pipeline` to `main`. Your PR description must include:

1. CV results table (all 5 models with all metrics – copy-paste or format as a Markdown table)
2. Final test-set metrics for your selected best model (accuracy, precision, recall, F1 from `final_evaluation`)
3. Recommendation paragraph (4–6 sentences connecting metrics to business context)
4. Paste your PR URL into TalentLMS -> Module 5 Week A -> Integration 5A to submit this assignment.

Note: Keep your `evaluation_pipeline.py` script clean and runnable. In Week B, you will extend this pipeline with decision tree and random forest models and produce the final Module 5 model comparison deliverable.